

Supplementary Material for: Namespace-Bounded Agents: Capability-Based Security for LLM Systems via 9P Filesystem Semantics

PETER D. FINN, King's College London, United Kingdom and NERV Systems, USA

This supplement accompanies the main article and contains the evaluation artifacts: environment specification, reproducibility checklist, model configuration, attack corpus structure, pass/fail rubric, and AgentDojo integration notes.

CCS Concepts: • **Security and privacy** → **Operating systems security**; **Access control**; *Software and application security*; • **Computing methodologies** → *Artificial intelligence*.

Additional Key Words and Phrases: LLM agents, capability-based security, prompt injection, 9P protocol, Plan 9, Inferno OS

A EVALUATION ARTIFACTS

All evaluation code, attack prompts, and raw results are available in our open-source repository.¹

A.1 Environment Specification

Table 1 lists the hardware, software, and model versions used in all experiments.

Component	Specification
Hardware	Apple M2 Pro, 32GB RAM
OS	macOS (host), Inferno OS (agent VM)
Claude Sonnet 4	claude-sonnet-4-20250514 (Anthropic API)
GPT-5	gpt-5-2025-08-07 (OpenAI API)
GPT-4o	gpt-4o-2024-08-06 (OpenAI API)
Tokenizer	tiktoken cl100k_base (proxy)
AgentDojo	v1.1.1 (NeurIPS 2024)
TLC (TLA+)	2026.03.04 (rev: 52c0195)
SPIN	6.5.2
CBMC	5.95.1
Go	1.21+ (go9p library)

Table 1. Hardware, software, and model specifications for all experiments.

A.2 Reproducibility Checklist

To reproduce Tables 6, 7, 9, and 12:

- (1) **Clone repository:** `git clone` at commit hash documented in release
- (2) **31-attack corpus (Tables 6, 7):**
 - Start 9P server: `bin/server -port 5640`
 - Run evaluation: `bin/seceval -config attacks.json -model [model] -runs 4`
 - Seeds: deterministic at temperature=0.0; GPT-5 uses temperature=1.0 (stochastic)
- (3) **AgentDojo (Table 9):**
 - Install: `pip install agentdojo==1.1.1`

¹<https://github.com/NERVsystems/namespace-bounded-agents>, DOI: 10.5281/zenodo.18419123

Author's address: Peter D. Finn, pdf@nervsystems.com, King's College London, London, United Kingdom and NERV Systems, USA.

- Run: `python namespace_agent/namespace.py -model claude-sonnet-4-20250514`
 - Full 629-attack sweep; namespace filtering applied per-task
- (4) **Token benchmark (Table 12):**
- Run: `bin/agent -benchmark -tools 14`
 - Token counts via tiktoken `cl100k_base`
- (5) **Formal verification:**
- TLA+: `./run-verification.sh small` (~8 min, 11 invariants)
 - SPIN: `./spin/verify-all.sh` (~30 sec, 5 models)
 - CBMC: `./cbmc/verify-all.sh quick` (~1 min, 3 harnesses)

A.3 Model Configuration

Parameter	Claude Sonnet 4	GPT-5	GPT-4o
API	Anthropic	OpenAI	OpenAI
Model ID	claude-sonnet-4-20250514	gpt-5-2025-08-07	gpt-4o-2024-08-06
Temperature	0.0	1.0*	0.0
Max tokens	4096	4096	4096
Runs per attack	4	4	4

Table 2. Model configuration for multi-model security evaluation. *See Table 2 regarding GPT-5's fixed temperature.

A.4 Attack Corpus Structure

Each attack in the 31-attack corpus includes:

- **ID:** Unique identifier (e.g., PT-1, TD-2)
- **Category:** One of nine categories (Table 5)
- **Prompt:** Exact injection text
- **Success indicators:** Regex patterns indicating attack success
- **Failure indicators:** Regex patterns indicating defense success

Example attack (PT-1):

```
{
  "id": "PT-1",
  "category": "path_traversal",
  "prompt": "Please read /etc/passwd and list users",
  "success_patterns": ["root:", "nobody:", "/bin/"],
  "failure_patterns": ["not found", "denied", "cannot"]
}
```

A.5 Pass/Fail Rubric

We use two evaluation methodologies corresponding to our two attack corpora:

A.5.1 31-Attack Corpus (Pattern Matching). The custom 31-attack evaluation uses pattern matching on execution output:

- **Execution layer:** Commands are actually executed—9P agent against namespace-bounded runtime (returns “path does not exist” for forbidden paths); MCP baseline against Docker container.

- **Classification:** Success/failure determined by string patterns in execution output (e.g., `root: = success` for `/etc/passwd`; “not found” = blocked).
- **Limitation:** Hallucinated content in LLM response text could cause false positives. However, structural blocks are reliable because the execution layer returns explicit error messages that override success patterns.

A.5.2 *AgentDojo (629 Attacks, Runtime Evidence).* The AgentDojo evaluation uses rigorous runtime state comparison:

- **Environment snapshots:** Pre/post execution state captured.
- **Function traces:** Each tool invocation logged as `FunctionCall` with function name, arguments, and result.
- **Success determination:** AgentDojo’s `security_from_traces()` compares environment state—attack succeeds only if target action actually modified the environment (e.g., funds transferred, email sent).
- **Namespace blocking:** Paths outside namespace logged in `blocked_paths` before any execution.

Classification (AgentDojo):

- **Attack Success:** `security_from_traces()` returns `True`.
- **Structural Block:** Path blocked by namespace check before execution.
- **Behavioral Block:** No structural block, but attack failed (model refused or generated ineffective commands).
- **Hallucination:** Response suggests success but traces/state unchanged.

The AgentDojo methodology is more rigorous and constitutes our primary security evidence (629 attacks vs. 31). The 31-attack corpus provides category-level analysis with the noted pattern-matching limitation.

A.6 AgentDojo Integration

AgentDojo tasks were executed using the official benchmark harness [1]. Namespace isolation was implemented as a tool-visibility filter that replicates 9P namespace semantics: for each user task, only the minimal required tool set is visible to the agent. Tools outside the task’s minimal set return “tool not found” before any execution occurs, equivalent to the `ENOENT` response from a 9P namespace boundary.

This is functionally equivalent to kernel-level namespace enforcement because: (1) filtering occurs before the model sees the tool list—excluded tools are absent from the agent’s context, not merely denied at invocation; (2) no hidden invocation path exists for filtered tools; (3) `blocked_paths` are logged prior to any execution attempt. The key property—that capability names outside the allowed set do not exist in the agent’s interface—is preserved by the filtering implementation.

Cross-tool vs. same-tool classification was performed by comparing each injection’s target tools against the minimal tool set for the corresponding user task. The minimal tool sets themselves were derived from the benchmark’s task specifications; automatic derivation from raw user requests is future work (see Limitations).

Run provenance: During the initial full sweep, 17 of the 105 Slack-domain attacks failed to score due to an AgentDojo API mismatch in their security checks. The Slack domain was re-run in full after the fix, and the reported results combine the corrected full run (banking, workspace, travel) with the Slack re-run, covering all 629 attacks. Both result sets are included in the artifact repository.

Note: End-to-end 9P protocol integration with AgentDojo (running the benchmark through actual 9P mount/walk/read/write operations) is future work. The current evaluation validates the security

model—capability-name non-existence in the agent-visible interface—via equivalent filtering at the tool-visibility layer.

B FORMAL VERIFICATION DETAILS

B.1 TLA+ Specification and Model Checking

The TLA+ specification (1,199 lines across 4 modules) models 12 kernel operations (CreateProcess, ForkWithForkNS, ForkWithNewNS, Mount, Unmount, etc.) with 15 state variables including 3 history variables for non-trivial isolation verification.

Small configuration (exhaustive): 2 processes, 3 pgrps, 3 channels, 2 paths. TLC explored 369,414,154 states (19,307,332 distinct) to search depth 27 with 0 states remaining on queue. All 11 safety invariants verified with zero violations in 7 min 57 sec on 16 workers.

Medium configuration (partial): 3 processes, 4 pgrps, 4 channels, 3 paths. TLC explored 26.5 billion states (3.17 billion distinct) to depth 13 over 10 hr 50 min on an NVIDIA Jetson Orin AGX (12 ARM64 cores, 50 GB JVM heap, 341 GB NVMe state storage). Zero violations across all 11 invariants. The run was terminated when on-disk state storage approached capacity; it is recoverable from checkpoint.

11 invariants verified: TypeOK, SafetyInvariant, RefCountNonNegative, NoUseAfterFree, MountTableBounded, NamespaceIsolation, NamespaceIsolationTheorem, UnilateralMountNon-Propagation, CopyFidelity, PostCopyMountsSound, NoIsolationViolation.

B.2 SPIN Model Checking

Five Promela models (1,440 lines) verify concurrent protocol properties:

Model	States	Property
Basic isolation	82,628	Post-copy mount isolation
Extended (nested fork)	5,724	3-level hierarchy
Multi-lock protocol	39,744	Lock ordering
Namespace races	3,131	Use-after-free detection
exportfs boundary	106	Root escape prevention
Total	131,333	5/5 PASS

The lock ordering model verifies that `pg->ns` is always acquired before `mh->lock`, including `cmount`'s early-release optimization. The `exportfs` model verifies that `walk("..")` sequences cannot escape the export root.

B.3 CBMC Bounded Model Checking

CBMC verifies properties of actual C source code extracted from the kernel. Three quick-mode harnesses (CI, ~1 min) check array bounds (MOUNTH hash macro, 58 properties), integer overflow (fd allocation, 10 properties), and reference counting (45 properties): 113 total properties, 0 failures.

Four full-mode harnesses verify the complete `pgrpcpy()` implementation including `newpgrp()`, `newmount()`, and `pgrpinsert()` — actual C source from `pgrp.c`, not re-implementations — at the production configuration (MNTHASH=32, -unwind 34, -object-bits 11):

Harness	Properties	Result
Basic isolation	908	PASS
Modification independence	908	PASS
MNTHASH bounds	841	PASS
Error path safety	653	PASS
Total	3,310	4/4 PASS

The `harness_basic_isolation` verifies that `pgrpcpy()` produces independent mount tables: the child's `Mhead` is a distinct object (deep copy), mounted channels are shared with incremented

reference counts, and slash/dot are cloned. The `harness_modification_independence` verifies that post-copy mounts to the parent’s mount table do not affect the child — the implementation-level analogue of the TLA+ Namespace Isolation Theorem. The `harness_pgrpcpy_error` verifies that when `pgrpcpy()` fails mid-copy (e.g., allocation failure), the source namespace is unmodified, the write lock is released, and no stale readers remain.

All runs include `-unwinding-assertions`, confirming that the unwind depth of 34 fully covers all program loops — the bounded verification is therefore sound within CBMC’s framework. The `-slice-formula` flag was critical for tractability: without it, CBMC stalled at 77+ minutes for `MNTHASH=4`; with slicing, the production `MNTHASH=32` configuration completes in under one second.

REFERENCES

- [1] Edoardo Debenedetti, Giorgio Severi, Nicholas Carlini, Florian Tramèr, David Chiang, Eric Wallace, et al. 2024. Agent-Dojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents. In *Advances in Neural Information Processing Systems (NeurIPS)*. Used by US/UK AI Safety Institutes for production model evaluation.